State management

17 min read Updated 18 May 2026 Topic 28 of 77

Bookmark Mark done

State Management — Interview Questions

Questions: 7 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft, Atlassian, Cred

---

Q 10.1 How do you decide between local state, lifted state, and global state?

Difficulty: Mid **Asked at:** Every company — this is the single most important state management question

Most state management problems aren't about choosing the right library. They're about choosing the right scope. The wrong scope creates either unnecessary complexity (everything in Redux when local state would do) or prop drilling hell (passing data through 8 components that don't use it).

The decision framework:

Local state (useState) — Does only this component need this data?

JSX Copy

```jsx
// Search input value — only the SearchBar needs it while typing
function SearchBar({ onSearch }) {
  const [query, setQuery] = useState('');

  return (
    <input
```

```
    />
  );
}
```

Keep state as local as possible. A modal's open/closed state, a form input's value while typing, an accordion's expanded state, a tooltip's visibility — these are local. No other component cares about them. Putting them in global state is unnecessary complexity.

Lifted state (shared parent) — Do 2-3 sibling components need the same data?

JSX Copy

```jsx
// Both ProductList and CartSummary need the selected items
function ProductPage() {
  const [selectedItems, setSelectedItems] = useState([]);

  return (
    <>
      <ProductList onSelect={(item) => setSelectedItems(prev => [...prev,
      <CartSummary items={selectedItems} />
    </>
  );
}
```

The state lives in the closest common ancestor and is passed down via props. This is the right solution when 2-3 nearby components need the same data. If you're passing through more than 2-3 levels, it's time to consider context or global state.

Context — Do many components across the tree need this data, but it changes infrequently?

JSX Copy

```jsx
// Theme, locale, auth user — read by many components, changes rarely
const ThemeContext = createContext('light');

function App() {
```

```
      <Layout />
    </ThemeContext.Provider>
  );
}
```

Context is free — it's built into React, no library needed. But it has a critical limitation: every component that consumes a context re-renders when the context value changes, even if the specific piece they use didn't change. This makes Context terrible for frequently changing data.

Global state (Zustand, Redux, Jotai) — Does data need to be accessed from many unrelated parts of the app AND it changes frequently?

JSX Copy

```jsx
// Shopping cart — accessed by header badge, product cards, checkout page,
// cart drawer, and it changes on every add/remove
const useCartStore = create((set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
  removeItem: (id) => set((state) => ({
    items: state.items.filter(i => i.id !== id),
  })),
}));
```

The quick decision flow:

JAVASCRIPT Copy

```
Does only this component need it?
  → Yes: useState (local)

  Do 2-3 nearby components need it?
  → Yes: Lift state to parent, pass via props

  Do many components need it, but it rarely changes?
  → Yes: Context (theme, auth, locale)
```

```
Does the data come from a server API?
→ Yes: Server state library (TanStack Query, SWR) — see Q10.5
```

Follow-up: "What's the most common state management mistake you've seen?"

Putting everything in global state. Teams adopt Redux on day one and store form input values, modal visibility, and tooltip state in the global store. This adds boilerplate, slows down development, and creates performance issues. Start local, escalate only when there's a real reason.

Q 10.2 What are the performance traps of React Context? When should you NOT use it?

Difficulty: Mid **Asked at:** Google, Flipkart, Razorpay, Amazon

Context's performance problem is straightforward: when the context value changes, every component that calls `useContext` for that context re-renders. There's no selector mechanism — you can't subscribe to just one field.

JSX Copy

```jsx
// THE PROBLEM
const AppContext = createContext();

function AppProvider({ children }) {
  const [user, setUser] = useState(null);
  const [theme, setTheme] = useState('light');
  const [notifications, setNotifications] = useState([]);

  // This object is recreated every render → every consumer re-renders
  return (
    <AppContext.Provider value={{ user, theme, notifications, setUser, set
      {children}
    </AppContext.Provider>
  );
}
```

```
function ThemeToggle() {
  const { theme, setTheme } = useContext(AppContext);
  // Re-renders when notifications change! It doesn't use notifications,
  // but the entire context value changed.
  return <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'li
}
```

Fix 1: Split contexts by change frequency

JSX Copy

```
// Separate contexts for data that changes at different rates
const AuthContext = createContext();      // Changes on login/logout (rarel
const ThemeContext = createContext();     // Changes on toggle (rarely)
const NotificationContext = createContext(); // Changes often

function App() {
  return (
    <AuthProvider>
      <ThemeProvider>
        <NotificationProvider>
          <Layout />
        </NotificationProvider>
      </ThemeProvider>
    </AuthProvider>
  );
}

// Now ThemeToggle only re-renders when ThemeContext changes
function ThemeToggle() {
  const { theme, setTheme } = useContext(ThemeContext);
  return <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'li
}
```

Fix 2: Memoize the context value

JSX Copy

```
  // Memoize so the value object reference stays stable
  // unless theme actually changes
  const value = useMemo(() => ({ theme, setTheme }), [theme]);

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Fix 3: Memoize consumers

JSX Copy

```
// Wrap expensive children with React.memo so they don't re-render
// when the parent re-renders due to context changes
const ExpensiveList = React.memo(function ExpensiveList({ items }) {
  return items.map(item => <ListItem key={item.id} item={item} />);
});
```

When NOT to use Context:

- Frequently changing data (every keystroke, every scroll event, every second) — use Zustand or Jotai instead, which support selectors
- Large state objects where different consumers need different slices — no way to subscribe to just one field
- When you need time-travel debugging, middleware, or devtools — Context doesn't support these

When Context is great:

- Theme, locale, auth status — read by many, changes rarely
- Feature flags — read everywhere, changes never during a session

Q 10.3 How does Redux Toolkit work? What's RTK Query?

Difficulty: Mid **Asked at:** Flipkart, Amazon, Swiggy, Walmart

Redux Toolkit (RTK) is modern Redux — it eliminates the boilerplate that made classic Redux painful (action types, action creators, switch statements in reducers, immutable update logic).

JAVASCRIPT Copy

```
import { configureStore, createSlice, createAsyncThunk } from '@reduxjs/to

// Async thunk — handles loading/success/error states automatically
const fetchProducts = createAsyncThunk(
  'products/fetch',
  async (category) => {
    const response = await fetch(`/api/products?category=${category}`);
    return response.json();
  }
);

const productsSlice = createSlice({
  name: 'products',
  initialState: {
    items: [],
    status: 'idle',  // 'idle' | 'loading' | 'succeeded' | 'failed'
    error: null,
  },
  reducers: {
    sortByPrice: (state) => {
      // Immer lets you "mutate" — it creates immutable updates behind the
      state.items.sort((a, b) => a.price - b.price);
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchProducts.pending, (state) => {
        state.status = 'loading';
```

```
        state.items = action.payload;
      })
      .addCase(fetchProducts.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.error.message;
      });
  },
});

const store = configureStore({
  reducer: { products: productsSlice.reducer },
});
```

RTK Query — server state management built into Redux:

RTK Query handles data fetching, caching, background refetching, and cache invalidation — similar to TanStack Query but integrated with Redux.

JAVASCRIPT Copy

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';

const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  tagTypes: ['Product'],
  endpoints: (builder) => ({
    getProducts: builder.query({
      query: (category) => `/products?category=${category}`,
      providesTags: ['Product'],
    }),
    updateProduct: builder.mutation({
      query: ({ id, ...patch }) => ({
        url: `/products/${id}`,
        method: 'PATCH',
        body: patch,
      }),
      invalidatesTags: ['Product'],  // Refetch product list after mutatio
    }),
  }),
```

```
// Auto-generated hooks
const { useGetProductsQuery, useUpdateProductMutation } = api;

// In a component
function ProductList({ category }) {
  const { data, isLoading, error } = useGetProductsQuery(category);
  // Automatic caching, deduplication, background refetching
}
```

When to choose Redux Toolkit:

- Large teams that need predictable state updates and enforced patterns
- Complex client-side state with many interrelated pieces
- When you need middleware (logging, analytics, persistence)
- When you need time-travel debugging (Redux DevTools)
- When the team already knows Redux

When Redux is overkill:

- Small to medium apps with simple state needs
- When most state is server data (TanStack Query handles this better standalone)
- When the team is small and doesn't need the structure Redux enforces

---

Q 10.4 What is Zustand? When does it beat Redux?

Difficulty: Mid **Asked at:** Razorpay, Cred, Swiggy, Flipkart

Zustand is a lightweight state management library (1KB gzipped) that provides global state with selectors, no boilerplate, and no providers.

JAVASCRIPT Copy

```
const useStore = create((set, get) => ({
  // State
  cart: [],
  total: 0,

  // Actions
  addItem: (item) => set((state) => {
    const cart = [...state.cart, item];
    return { cart, total: cart.reduce((sum, i) => sum + i.price, 0) };
  }),

  removeItem: (id) => set((state) => {
    const cart = state.cart.filter(i => i.id !== id);
    return { cart, total: cart.reduce((sum, i) => sum + i.price, 0) };
  }),

  clearCart: () => set({ cart: [], total: 0 }),

  // Derived values using get()
  getItemCount: () => get().cart.length,
}));
```

Usage — with automatic selectors (no unnecessary re-renders):

JSX Copy

```
function CartBadge() {
  // Only re-renders when cart.length changes — not when total changes
  const count = useStore((state) => state.cart.length);
  return <span className="badge">{count}</span>;
}

function CartTotal() {
  // Only re-renders when total changes
  const total = useStore((state) => state.total);
  return <span>₹{total}</span>;
}

function AddButton({ item }) {
  // Subscribes to nothing — never re-renders due to state changes
```

Why Zustand beats Redux in many cases:

No providers wrapping your app. No action types, no action creators, no reducers, no dispatch. Selectors built in (component only re-renders when its selected slice changes). Works outside React (in vanilla JS, in event handlers, in utility functions). 1KB vs ~11KB for Redux Toolkit.

Zustand with middleware:

JAVASCRIPT Copy

```
import { create } from 'zustand';
import { persist, devtools } from 'zustand/middleware';

const useStore = create(
  devtools(
    persist(
      (set) => ({
        theme: 'light',
        setTheme: (theme) => set({ theme }),
      }),
      { name: 'app-settings' }  // Persists to localStorage
    )
  )
);
```

When to choose Zustand over Redux:

- Small to medium apps where Redux's ceremony isn't justified
- When you want global state with zero boilerplate
- When you need state outside React (Web Workers, event handlers, tests)
- When bundle size matters

When Redux still wins:

- When the team already has Redux expertise and tooling

Follow-up: "What about Jotai and Recoil?"

Jotai (atomic state) and Recoil (also atomic) take a bottom-up approach — you define individual atoms of state and derive computed values from them. This is great when state is highly granular (many independent pieces). Zustand is top-down (one store, multiple slices) — better when state is cohesive and actions affect multiple pieces together. Most teams find Zustand simpler to reason about.

Q 10.5 What's the difference between server state and client state? How does TanStack Query change the picture?

Difficulty: Senior **Asked at:** Google, Razorpay, Flipkart, Swiggy, Amazon

This question reveals the biggest mental model shift in state management over the past few years. Most state in a typical frontend app isn't client state at all — it's cached server data. And managing cached server data with Redux or Zustand is solving the wrong problem.

Client state — originates in the browser, owned by the frontend:

- UI state: modal open/closed, active tab, sidebar collapsed
- Form state: input values, validation errors, dirty tracking
- User preferences: theme, locale, layout density

Server state — originates on the server, the frontend has a cached copy:

- User profile data
- Product listings
- Order history

The difference matters because server state has properties that client state doesn't: it can become stale, another user can modify it, it needs to be synchronized across tabs, and it requires loading/error handling for every access.

The problem with managing server state in Redux/Zustand:

JAVASCRIPT Copy

```
// Classic approach - you manage everything manually
const productsSlice = createSlice({
  name: 'products',
  initialState: { items: [], loading: false, error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchProducts.pending, (state) => { state.loading = true; })
      .addCase(fetchProducts.fulfilled, (state, action) => {
        state.loading = false;
        state.items = action.payload;
      })
      .addCase(fetchProducts.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      });
  },
});
// You write this for EVERY API call. Loading state, error state, caching,
// refetching, deduplication, invalidation - all manual.
```

TanStack Query makes server state declarative:

JSX Copy

```
function ProductList({ category }) {
  const { data, isLoading, error, refetch } = useQuery({
    queryKey: ['products', category],
    queryFn: () => fetch(`/api/products?category=${category}`).then(r => r
```

```
  });

  if (isLoading) return <Skeleton />;
  if (error) return <ErrorState retry={refetch} />;
  return data.map(p => <ProductCard key={p.id} product={p} />);
}
```

What TanStack Query handles automatically:

- **Caching:** Same query key → returns cached data instantly
- **Deduplication:** 10 components request the same data → 1 API call
- **Background refetching:** Stale data is shown immediately, fresh data replaces it when ready
- **Automatic retry:** Failed requests retry with exponential backoff
- **Window focus refetching:** Data refreshes when user returns to the tab
- **Garbage collection:** Unused cache entries are cleaned up automatically
- **Optimistic updates:** Show the result immediately, roll back if the mutation fails

Mutations with cache invalidation:

JSX Copy

```jsx
function useUpdateProduct() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: (product) => fetch(`/api/products/${product.id}`, {
      method: 'PATCH',
      body: JSON.stringify(product),
    }),
    onSuccess: () => {
      // Invalidate the products cache — forces refetch
      queryClient.invalidateQueries({ queryKey: ['products'] });
    },
```

The modern state management architecture:

JAVASCRIPT Copy

```
Client state (UI, forms, preferences)  →  Zustand or useState/useReducer
  Server state (API data)                  →  TanStack Query or SWR
  URL state (filters, pagination, tabs)    →  URL params (useSearchParams)
  Form state (complex forms)               →  React Hook Form or Formik
```

Most applications need very little global client state once server state is handled by TanStack Query. The cart, the theme, and maybe a couple of UI flags — that's it.

Follow-up: "What about SWR vs TanStack Query?"

SWR (by Vercel) is simpler and lighter — great for basic data fetching with stale-while-revalidate. TanStack Query is more powerful — it supports mutations, optimistic updates, infinite queries, query cancellation, and more complex cache management. For most production applications, TanStack Query's extra features justify the slightly larger API surface.

Q 10.6 What are state machines? When would you use XState over useState?

Difficulty: Senior **Asked at:** Google, Razorpay, Amazon

Covered conceptually in Chapter 9 (Q9.13). Here we focus on when to reach for a state machine over plain React state.

The problem with boolean flags:

JSX Copy

```jsx
// Payment flow with booleans — impossible states are possible
const [isLoading, setIsLoading] = useState(false);
```

```
// Can you have isLoading: true AND isSuccess: true? The code allows it.
// Can you have isError: true AND isRetrying: false when retry was clicked
// Every new flag doubles the possible states. 4 booleans = 16 possible st
// most of which are invalid.
```

A state machine makes impossible states impossible:

JAVASCRIPT Copy

```javascript
import { createMachine, assign } from 'xstate';

const paymentMachine = createMachine({
  id: 'payment',
  initial: 'idle',
  context: { error: null, retryCount: 0 },
  states: {
    idle: {
      on: { SUBMIT: 'processing' },
    },
    processing: {
      invoke: {
        src: 'processPayment',
        onDone: 'success',
        onError: {
          target: 'failed',
          actions: assign({ error: (_, event) => event.data.message }),
        },
      },
    },
    failed: {
      on: {
        RETRY: {
          target: 'processing',
          guard: ({ context }) => context.retryCount < 3,
          actions: assign({ retryCount: (ctx) => ctx.retryCount + 1 }),
        },
        CANCEL: 'idle',
      },
```

```
    },
  },
});
```

The machine is in exactly one state at a time. From `processing`, it can only go to `success` or `failed` — never both. From `failed`, you can retry (max 3 times) or cancel. The `SUBMIT` event only works in `idle` state — clicking the button twice during processing does nothing.

Using in React:

JSX Copy

```jsx
import { useMachine } from '@xstate/react';

function PaymentForm() {
  const [state, send] = useMachine(paymentMachine);

  return (
    <div>
      {state.matches('idle') && (
        <button onClick={() => send({ type: 'SUBMIT' })}>Pay Now</button>
      )}
      {state.matches('processing') && <Spinner />}
      {state.matches('failed') && (
        <>
          <p>Payment failed: {state.context.error}</p>
          <button onClick={() => send({ type: 'RETRY' })}>Retry</button>
          <button onClick={() => send({ type: 'CANCEL' })}>Cancel</button>
        </>
      )}
      {state.matches('success') && <p>Payment successful!</p>}
    </div>
  );
}
```

When to use XState / state machines:

- Complex flows with many states and transitions (checkout, onboarding wizards)

- When you need to visualize the flow (XState Visualizer generates diagrams from your machine definition)
- When business logic needs to be separated from UI logic

When useState is fine:

- Simple toggles (open/closed, on/off)
- Linear forms without complex branching
- State with 2-3 possible values and obvious transitions

Q 10.7 How do you handle state synchronization — optimistic updates, conflict resolution, and offline?

Difficulty: Senior **Asked at:** Google, Amazon, Flipkart, Razorpay

Optimistic updates — show the result before the server confirms:

JSX Copy

```jsx
function useLikePost(postId) {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: () => fetch(`/api/posts/${postId}/like`, { method: 'POST'

    // Optimistically update the cache before the server responds
    onMutate: async () => {
      await queryClient.cancelQueries({ queryKey: ['post', postId] });
      const previous = queryClient.getQueryData(['post', postId]);

      queryClient.setQueryData(['post', postId], (old) => ({
        ...old,
        liked: true,
        likeCount: old.likeCount + 1,
      }));
```

```
    // Rollback on error
    onError: (err, vars, context) => {
      queryClient.setQueryData(['post', postId], context.previous);
    },

    // Refetch to ensure consistency
    onSettled: () => {
      queryClient.invalidateQueries({ queryKey: ['post', postId] });
    },
  });
}
```

Conflict resolution — when two users edit the same data:

Last-write-wins is the simplest approach. Whoever saves last overwrites the previous version. Simple but loses data.

Version-based (optimistic locking): Each record has a version number. When saving, the client sends the version it loaded. If the server's version is different (someone else edited), the save is rejected and the client must merge or reload.

JAVASCRIPT Copy

```
async function saveDocument(doc) {
  const response = await fetch(`/api/docs/${doc.id}`, {
    method: 'PUT',
    body: JSON.stringify({ ...doc, expectedVersion: doc.version }),
  });

  if (response.status === 409) {
    // Conflict — someone else modified the document
    const serverVersion = await response.json();
    // Show diff UI, let user merge, or reload with server version
    return handleConflict(doc, serverVersion);
  }
}
```

collaborative editing tools. Libraries like Yjs and Automerge implement CRDTs for JavaScript.

Offline state synchronization:

JAVASCRIPT

Copy

```
// Queue actions while offline, replay when online
class OfflineQueue {
  constructor() {
    this.queue = JSON.parse(localStorage.getItem('offlineQueue') || '[]');
  }

  add(action) {
    this.queue.push({ ...action, timestamp: Date.now() });
    localStorage.setItem('offlineQueue', JSON.stringify(this.queue));
  }

  async replay() {
    while (this.queue.length > 0) {
      const action = this.queue[0];
      try {
        await executeAction(action);
        this.queue.shift();
        localStorage.setItem('offlineQueue', JSON.stringify(this.queue));
      } catch (err) {
        if (isConflict(err)) {
          // Handle conflict for this specific action
          await resolveConflict(action, err);
          this.queue.shift();
        } else {
          break;  // Network error - stop replaying, try again later
        }
      }
    }
  }
}

// Listen for online event
window.addEventListener('online', () => offlineQueue.replay());
```

JAVASCRIPT

Copy

```javascript
// Using BroadcastChannel to sync state across tabs
const channel = new BroadcastChannel('app-state');

// When state changes in one tab, broadcast it
useStore.subscribe((state) => {
  channel.postMessage({ type: 'STATE_UPDATE', state });
});

// When another tab broadcasts, update local state
channel.onmessage = (event) => {
  if (event.data.type === 'STATE_UPDATE') {
    useStore.setState(event.data.state);
  }
};
```

Follow-up: "When would you build offline support vs just showing an error?"

Build offline support when: users expect it (note-taking apps, task managers), users are frequently on unreliable networks (mobile apps in emerging markets), or the cost of data loss is high (document editors, form submissions). Show an error when: the feature requires real-time server validation (payment processing), or the offline implementation cost outweighs the user benefit (admin dashboards used on stable office networks).

QUICK CHECK — no marks, just for you.

1. A modal's open/closed flag, an accordion's expanded flag, and a tooltip's visibility all belong in which scope, according to the lesson's decision framework?

- ○ Context, so any component can toggle them later
- ○ Local component state with useState
- ○ A global Zustand or Redux store, since UI state should be centralized
- ○ Lifted to the top-level App component and passed down via props

2. One AppContext provider holds user, theme, and notifications in a single value object. Notifications update frequently. Why does a ThemeToggle that only reads theme still re-render on every notification update?

- ○ Because useContext always re-renders its component on every parent render, regardless of the context value
- ○ Because React re-renders context consumers in provider order, so earlier consumers re-run when later ones update
- ○ Because setNotifications is included in the value object, and state setters are recreated on every render
- ○ Because any component calling useContext re-renders when the context value changes, with no way to subscribe to a single field

Check answer

3. In the Zustand cart store, AddButton does `const addItem = useStore((state) => state.addItem)`. An item is added, changing both cart and total. What happens to AddButton?

- ○ It re-renders only once on the first add, then Zustand memoizes it for later updates
- ○ It re-renders, because addItem is recreated by set() on each update
- ○ It does not re-render, because its selector returns only the action, which is not changing state it subscribes to
- ○ It re-renders, because any set() call re-renders every component using the store

Check answer

Previous
Architectural patterns

Next
Foundations

LEARN MORE

ARTICLES & DOCS

- Managing State (react.dev)
  react.dev
- TanStack Query (server state)
  github.com

VIDEOS



What is state management?
YouTube



State management concepts — just enough
YouTube



Benefits of state management & patterns
YouTube

Hand-picked references for this lesson.